

Neural Network with Batch Normalization from scratch

Vu Anh Ha

April 2026

1 Introduction

Batch normalization is a method to address the phenomenon of "Internal Covariate Shift" in neural network - the change in each layer inputs' distribution slows down the training and make it harder to train models with saturating nonlinearities - and simultaneously solve the problem of vanishing gradient.

This is my best attempt to, from my best of understanding, rebuild the whole architecture of neural network from scratch, step by step and laying all the math behind it step by step as an outline for me to implement it in numpy later.

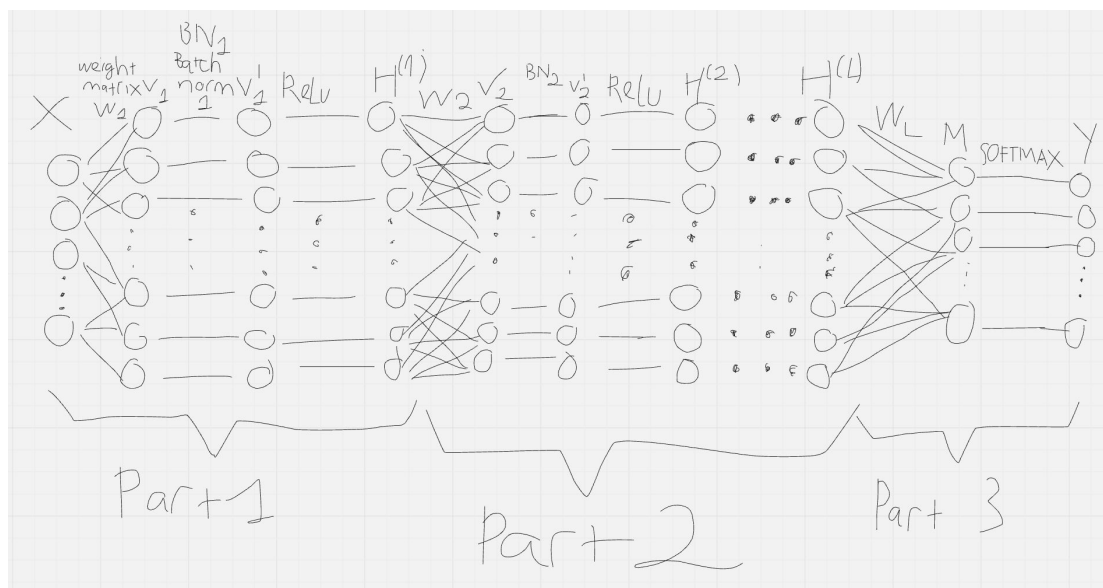


Figure 1: Sketch of neural network with batch normalization

2 Theory and theorem of Matrix and Multi-variative Derivative that I will use

1. Consider any square matrix $W \in R^{d \times d}$ then the trace of W is the diagonal sum of W or formally:

$$Tr(W) = \sum_{i=1}^d W_{i,i}.$$

One of the most important property of Trace that I will use is that $Tr(AB) = Tr(BA)$ because from it we can derive: $Tr(ABC) = Tr(A(BC)) = Tr((BC)A) = Tr(BCA) = Tr(B(CA)) = Tr((CA)B) = Tr(CAB)$ or we can freely cycle terms in Trace.

Proof: Let $A \in R^{d \times k}$, $B \in R^{k \times d}$ then $W = AB \in R^{d \times d}$.

$$\begin{aligned} Tr(AB) &= Tr(W) = \sum_{i=1}^d W_{i,i} = \sum_{i=1}^d \sum_{j=1}^k A_{i,j} B_{j,i} (*) \\ &= \sum_{i=1}^d \sum_{j=1}^k B_{j,i} A_{i,j} = \sum_{j=1}^k \sum_{i=1}^d B_{j,i} A_{i,j} = \sum_{j=1}^k (BA)_{j,j} = Tr(BA). \end{aligned}$$

Or in other way $Tr(AB) = Tr(BA)$ since it is the sum of all possible unique multiplication of entries from A and B .

Also, another property of trace is that: Since from (*) we get

$$Tr(AB) = \sum_{i=1}^d \sum_{j=1}^k A_{i,j} B_{j,i} = \sum_{i=1}^d \sum_{j=1}^k A_{j,i}^T B_{j,i}$$

Since A^T and B are matrices with the same size, we get the following property:

With A, B being matrix with the same size of $m \times n$ then: $Tr(A^T B) = \sum_{i=1}^m \sum_{j=1}^n A_{i,j} B_{i,j}$

2. Let $f : R^{m \times n} \rightarrow R$ be a function that map a matrix W to a scalar.,
or $f(W) = f(W_{1,1}, W_{1,2}, \dots, W_{m,n}) = y$ then $\frac{df}{dW}$ is the matrix of the same size as W :

$$\frac{df}{dW} = \begin{bmatrix} \frac{df}{dW_{1,1}} & \frac{df}{dW_{1,2}} & \cdots & \frac{df}{dW_{1,n}} \\ \frac{df}{dW_{2,1}} & \frac{df}{dW_{2,2}} & \cdots & \frac{df}{dW_{2,n}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{df}{dW_{m,1}} & \frac{df}{dW_{m,2}} & \cdots & \frac{df}{dW_{m,n}} \end{bmatrix}$$

For example, let $f(W) = \sum_{i=1}^m \sum_{j=1}^n W_{i,j}^2$ then $\frac{df}{dW_{i,j}} = 2W_{i,j}$

$$\text{Hence, } \frac{df}{dW} = \begin{bmatrix} 2W_{1,1} & 2W_{1,2} & \cdots & 2W_{1,n} \\ 2W_{2,1} & 2W_{2,2} & \cdots & 2W_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ 2W_{m,1} & 2W_{m,2} & \cdots & 2W_{m,n} \end{bmatrix} = 2W$$

3. $df = Tr((\frac{df}{dW})^T dW)$ or if $df = Tr(A^T dW)$ then $A = \frac{df}{dW}$

Proof: Consider the same f function in 2.,

By definition, $dW \in R^{m \times n}$ is a matrix such that:

$$dW = \begin{bmatrix} dW_{1,1} & dW_{1,2} & \cdots & dW_{1,n} \\ dW_{2,1} & dW_{2,2} & \cdots & dW_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ dW_{m,1} & dW_{m,2} & \cdots & dW_{m,n} \end{bmatrix} \quad \text{Furthermore, } df = \sum_{i=1}^m \sum_{j=1}^n \frac{df}{dW_{i,j}} \cdot dW_{i,j} =$$

$$\sum_{i=1}^m \sum_{j=1}^n \left(\frac{df}{dW} \right)_{i,j} \cdot dW_{i,j}$$

Applying property of trace from 1. we get $df = \text{Tr}\left(\left(\frac{df}{dW}\right)^T dW\right)$, as desired.

4. Let $A, B_1, B_2, \dots, B_n$ be matrices such that $A = B_1 B_2 \dots B_n$, then

$$dA = (dB_1)B_2 \dots B_n + B_1(dB_2)B_3 \dots B_n + \dots + B_1 B_2 \dots B_{(n-1)}(dB_n)$$

We can consider $A = f(B_1, B_2, \dots, B_n)$, hence by the rule of derivative:

$$dA = \lim_{t \rightarrow 0} \frac{f(B_1 + td_{B_1}, B_2 + td_{B_2}, \dots, B_n + td_{B_n}) - f(B_1, B_2, \dots, B_n)}{t}$$

$$= \frac{(B_1 + td_{B_1})(B_2 + td_{B_2}) \dots (B_n + td_{B_n}) - B_1 B_2 \dots B_n}{t}$$

$$\begin{aligned}
&= \lim_{t \rightarrow 0} \frac{1}{t} [t(dB_1B_2...B_n + B_1dB_2B_3...B_n + ... + B_1B_2...B_{n-1}dB_n) \\
&+ t^2(dB_1dB_2B_3...B_n + B_1dB_2dB_3B_4...B_n + ... + B_1B_2...B_{n-2}dB_{n-1}dB_n) \\
&+ t^3(g_3(B_1, ..., B_n, dB_1, ..., dB_n)) + ... + t^n g_n(B_1, ..., B_n, dB_1, ..., dB_n)] \\
&= (dB_1B_2...B_n + B_1dB_2B_3...B_n + ... + B_1B_2...B_{n-1}dB_n) \\
&+ \lim_{t \rightarrow 0} t(g_2(B_1, ..., B_n, dB_1, ..., dB_n) + ... + g_n(B_1, ..., B_n, dB_1, ..., dB_n)) = dB_1B_2...B_n + \\
&B_1dB_2B_3...B_n + ... + B_1B_2...B_{n-1}dB_n, \text{ as desired}
\end{aligned}$$

5. Example of Application of the above properties in backpropagation

Assume we have the loss function L , by backpropagating we can calculate $\frac{dL}{dZ}$, and let's assume $Z = WX$ then the next step of propagation we need to calculate $\frac{dL}{dW}$ and $\frac{dL}{dX}$ in term of $\frac{dL}{dZ}$. Starting with $\frac{dL}{dW}$, in this case X is a constant matrix hence $dX = 0$ Applying

$$3. \text{ we get } dL = Tr((\frac{dL}{dZ})^T dZ).$$

$$\text{Applying 4. we get } Z = (dW)X + W(dX) = (dW)X$$

$$\text{Hence } dL = Tr((\frac{dL}{dZ})^T dZ) = Tr((\frac{dL}{dZ})^T (dW)X)$$

Applying property of trace from 1., we get

$$dL = Tr(X(\frac{dL}{dZ})^T dW) = Tr((X^T)^T (\frac{dL}{dZ})^T dW) = Tr(\frac{dL}{dZ} X^T)^T dW$$

$$(\text{since } A^T B^T = (BA)^T) \text{ Hence } \frac{dL}{dW} = \frac{dL}{dZ} X^T \text{ Similarly, } \frac{dL}{dX} = W^T (\frac{dL}{dZ})$$

Hence, having calculated $\frac{dL}{dZ}$, we can apply the above properties like above to

calculate $\frac{dL}{dW}$, $\frac{dL}{dX}$ as the next step of the backpropagation.

3 Training Breakdown

Let $Epochs$ be the number of times we will train our model throughout the whole batch in minibatches, B be the size of the mini-batch, N be the number of mini-batches in the whole X_{train}

for i in range($Epochs$):

Sample(X_{train})

Let X_{train} be a set of $X \in R^{d*1}$, let $mean(X_i)$ and $std(X_i)$ be the mean and standard deviation of every i -th element of every X respectively with $i \in \overline{1, d}$.

Then for every element X_i of X , we normalize them: $X_i \leftarrow \frac{X_i - mean(X_i)}{std(X_i)}$

Split X_{train} into N number of mini-batches size B

for j is range(N):

train(X)

Let d be the dimension of each vector X_i , $X \in R^{d*B}$ being the input matrix of a mini-batch.

Consider the process from X to the first hidden layer:

$$\text{Part 1: } X \xrightarrow{\text{WeightMatrix}} A \xrightarrow{\text{BatchNormalization}} B \xrightarrow{\text{ReLU}} H \xrightarrow{\text{Dropout}} H$$

with $H \in R^{k*B}$ being the first hidden layer.

We repeat the following process from this hidden layer to the next hidden layer:

$$\text{Part 2: } H^{(i)} \xrightarrow{\text{WeightMatrix}W^{(i)}} A \xrightarrow{\text{BatchNormalization}} B \xrightarrow{\text{ReLU}} H^{(i+1)} \xrightarrow{\text{Dropout}} H^{(i+1)}$$

The final hidden layer to the loss function:

$$\text{Part 3: } H^{(\text{last*layer})} \xrightarrow{\text{WeightMatrix}W} M \xrightarrow{\text{SoftMax}} Y$$

We calculate the final mini-batch loss L then we do: $\text{Step4} : \text{Backprop}_{\text{part3}} \Rightarrow \text{Step5} : \text{Backprop}_{\text{part2}} \Rightarrow \text{Step6} : \text{Backprop}_{\text{part1}}$ to finish $\text{train}(X)$

4 Remarks: Why we don't need to use bias to add to weighted sum between 2 hidden layers if our neural network use batchnorm

: As we know, the standard neural network is $H_{n+1} = WH_n + B = M + B$ with B being a matrix of bias β .

Apply batch norm: Consider any mean of the i -th row within a batch:

$$\text{batchmean}_i = (\sum \frac{1}{C}(M_j + B)) = B + \frac{1}{C} \sum M_j. \text{ Therefore } \text{batchstd}_i = \frac{1}{C} \sum \sqrt{\sum ((H_{n+1})_j - \text{batchmean}_i)^2} = \frac{1}{C} \sum \sqrt{\sum (M_j - \frac{1}{C} \sum M_j)^2}.$$

Therefore, batchnorm cancels out the bias.

5 Step 1: Feed forward in Part 1 and Step 6: Backpropagation happening inside Part 1

TUNEABLE PARAMETER: dropout rate α , learning rate c .

STEP 1: FEED FORWARD IN PART 1

$$X \in R^{d*B} \text{ and } X = \begin{bmatrix} | & | & \dots & | \\ X^{(1)} & X^{(2)} & \dots & X^{(B)} \\ | & | & \dots & | \end{bmatrix}$$

$$\text{Let } W \in R^{k*d} \text{ be the weight matrix such that: } W = \begin{bmatrix} - & W_1 & - \\ - & W_2 & - \\ & \vdots & \\ - & W_k & - \end{bmatrix}$$

$M = WX$ then $M \in R^{k*B}$ and

$$M = \begin{bmatrix} W_1X^{(1)} & W_1X^{(2)} & \dots & W_1X^{(B)} \\ W_2X^{(1)} & W_2X^{(2)} & \dots & W_2X^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ W_kX^{(1)} & W_kX^{(2)} & \dots & W_kX^{(B)} \end{bmatrix}$$

According to batch normalization, let $batchmean_i$ be the mean of $row_i(M)$ and $batchSTD_i$ be the standard deviation of $row_i(M)$ then for every entry $M_{i,j}$ of Z :

$$M_{i,j} \leftarrow \frac{M_{i,j} - batchmean_i}{batchSTD_i}$$

$$\begin{aligned} \text{We see that: } batchmean_i &= \frac{1}{B} \cdot (W_iX^{(1)} + W_iX^{(2)} + \dots + W_iX^{(B)}) \\ &= \frac{1}{B} \cdot W_i(X^{(1)} + X^{(2)} + \dots + X^{(B)}) \end{aligned}$$

$$\text{Denote } E_X = \frac{1}{B} \cdot (X^{(1)} + X^{(2)} + \dots + X^{(B)})$$

† Let $X'^{(i)} = X^{(i)} - E_X$ for all i in $1, 2, \dots, B$.

then for all entry: $M_{i,j}$ of M , we have $M_{i,j} - batchmean_i = W_iX^{(j)} - W_iE_X = W_i(X^{(j)} - E_X) = W_iX'^{(j)}$.

Let v_i be the variance of row i of M then we have $\frac{1}{batchSTD_i} = (v_i + \epsilon)^{-1/2}$
(We set $\epsilon = 10^{-7}$ which is small enough to not affect anything considerably to the value of standard deviation while ensure that the denominator will not receive invalid value like NaN when v_i tends to 0.)

Furthermore notice that $v_i = \sum_{j=1}^B (M_{i,j} - batchmean_i)^2 = \sum_{j=1}^B (W_iX'^{(j)})^2$ Then, denote N as the matrix M after batch normalization, we have

$$N = \begin{bmatrix} W_1X'^{(1)}(v_1 + \epsilon)^{-1/2} & W_1X'^{(2)}(v_1 + \epsilon)^{-1/2} & \dots & W_1X'^{(B)}(v_1 + \epsilon)^{-1/2} \\ W_2X'^{(1)}(v_2 + \epsilon)^{-1/2} & W_2X'^{(2)}(v_2 + \epsilon)^{-1/2} & \dots & W_2X'^{(B)}(v_2 + \epsilon)^{-1/2} \\ \vdots & \vdots & \ddots & \vdots \\ W_kX'^{(1)}(v_k + \epsilon)^{-1/2} & W_kX'^{(2)}(v_k + \epsilon)^{-1/2} & \dots & W_kX'^{(B)}(v_k + \epsilon)^{-1/2} \end{bmatrix}$$

Let $f : R \rightarrow R$ such that $f(x) = (x + \epsilon)^{-1/2}$,

$$\text{Let } X' = \begin{bmatrix} | & | & & | \\ X'^{(1)} & X'^{(2)} & \dots & X'^{(B)} \\ | & | & & | \end{bmatrix}, V = \begin{bmatrix} f(v_1) & 0 & \dots & 0 \\ 0 & f(v_2) & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & f(v_k) \end{bmatrix}$$

Then we get $N = VWX'$ with $V \in V^{k*k}$, $W \in V^{k*d}$, $X' \in d * B$, $N \in R^{k*B}$.
Next, we perform affine transformation on this layer: $N' = \gamma N + \beta$

Next, we pass this layer through ReLu let $g(x) = ReLu(x)$ then we get first hidden layer $H \in R^{k*B}$ such that $H_{i,j} = g(N'_{i,j})$ for every entry of H ,

Or we will denote as $G : R^{k*B} \rightarrow R^{k*B}$ such that $H = G(N')$

We dropout with the rate of α to finally get the true next hidden layer:

Let $H' \in R^{k*B}$ such that:

$P(H'_{i,j} = 0) = \alpha, P(H'_{i,j} = H_{i,j}) = 1 - \alpha$ for entry $H_{i,j}$ of H .

After that we scale up the remaining $1 - \alpha$ neurons: $H' = H' \cdot \frac{1}{1 - \alpha}$ to keep the strength of the signal. Because otherwise, the model has to adjust its parameter to handle the weaker signal.

STEP 6: BACKPROPAGATION IN PART 1

At this step, by backpropagation in earlier steps we get the value of $\frac{dL}{dH'}$

From the feedforward in step 1, the structure in part 1 is:

$$X \rightarrow X' \xrightarrow{VWX'} N \xrightarrow{\gamma N + \beta} N' \xrightarrow{ReLU(N')} H \xrightarrow{Dropout} H'$$

1. Calculate $\frac{dL}{dH}$ in terms of $\frac{dL}{dH'}$.

Note that if $H'_{i,j} = 0$ then we can see that $H_{i,j}$ doesn't contribute anything to L hence its gradient is zero.

Hence $(\frac{dL}{dH})_{i,j} = \frac{1}{1 - \alpha} \cdot (\frac{dL}{dH'})_{i,j}$ if $H'_{i,j} \neq 0$ and $(\frac{dL}{dH})_{i,j} = 0$ if $H'_{i,j} = 0$

2. Calculate $\frac{dL}{dN'}$ in terms of $\frac{dL}{dH}$.

$\frac{dL}{dN'_{i,j}} = \frac{dL}{dH_{i,j}} \cdot \frac{dH_{i,j}}{dN'_{i,j}} = \frac{dL}{dH_{i,j}} \cdot g'(N'_{i,j})$. Since $g(x) = \max(0, x)$, $g'(x) = 0$ if

$x < 0$ and $g'(x) = 1$ if $x \geq 0$, hence $(\frac{dL}{dN'})_{i,j} = \frac{dL}{dN'_{i,j}} = \frac{dL}{dH_{i,j}}$ if $H_{i,j} \neq 0$

and $(\frac{dL}{dN'})_{i,j} = \frac{dL}{dN'_{i,j}} = 0$ if $H_{i,j} = 0$ for every entry $H_{i,j}$ of H

3. Calculate $\frac{dL}{dN}, \frac{dL}{d\gamma}, \frac{dL}{d\beta}$ in terms of $\frac{dL}{dN'}$ and update β, γ .

$$\frac{dL}{dN} = \frac{dL}{dN'} \cdot \frac{dN'}{dN} = \gamma \frac{dL}{dN'}; dL = \sum_{i=1}^k \sum_{j=1}^B \frac{dL}{dN'_{i,j}} dN'_{i,j} = \sum_{i=1}^k \sum_{j=1}^B \frac{dL}{dN'_{i,j}} N_{i,j} \cdot d\gamma$$

$$\text{hence } \frac{dL}{d\gamma} = \sum_{i=1}^k \sum_{j=1}^B \frac{dL}{dN'_{i,j}} N_{i,j} = \text{Tr}((\frac{dL}{dN'})^T \cdot N). \text{ Similarly, } \frac{dL}{d\beta} = \sum_{i=1}^k \sum_{j=1}^B \frac{dL}{dN'_{i,j}}$$

We update the parameters: $\gamma \leftarrow \gamma - c \frac{dL}{d\gamma}, \beta \leftarrow \beta - c \frac{dL}{d\beta}$

4. Calculate $\frac{dL}{dW}$ in terms of $\frac{dL}{dN}$ and update the weight matrix W .

In this case V relies on W since v_i relies on W and X' is constant therefore $dX' = 0, dV \neq 0, dW \neq 0$.

$$N = VWX' \text{ hence we have } dN = (dV)WX' + V(dW)X' + VW(dX') = (dV)WX' + V(dW)X'$$

Hence, we have: $dL = Tr((\frac{dL}{dN})^T)dN = Tr((\frac{dL}{dN})^T)(dV)WX' + Tr((\frac{dL}{dN})^T)V(dW)X'$

We have:

$$dL_1 = Tr((\frac{dL}{dN})^T)V(dW)X' = Tr(X'((\frac{dL}{dN})^T)V(dW))$$

Since V is a diagonal matrix, we get $V^T = V$ hence

$$\begin{aligned} dL_1 &= Tr(X'((\frac{dL}{dN})^T)V^T(dW)) = Tr(((X')^T)^T(V\frac{dL}{dN})^T(dW)) \\ &= Tr((V\frac{dL}{dN}(X')^T)^T(dW)) \text{ hence } \frac{dL_1}{dW} = V\frac{dL}{dN}(X')^T(1) \end{aligned}$$

$$dL_2 = Tr((\frac{dL}{dN})^T)(dV)(WX') = Tr((WX')(\frac{dL}{dN})^T)(dV))$$

Let $P = WX'(\frac{dL}{dN})^T$ then $P \in R^{k \times k}$ and

$$dL_2 = \sum_{i=1}^k P_{i,i}(dV_{i,i}) = \sum_{i=1}^k P_{i,i}df(v_i).$$

As proven in step 1, $v_i = \sum_{j=1}^B (W_i X'^{(j)})^2 = \sum_{j=1}^B (\sum_{l=1}^d W_{i,l} X'_l{}^{(j)})^2$

hence for all $i = \overline{1, k}, l = \overline{1, d}$, we have

$$\begin{aligned} \frac{dv_i}{dW_{i,l}} &= \sum_{j=1}^B 2(W_i X'^{(j)}) \cdot X'_l{}^{(j)} = \sum_{j=1}^B 2(WX')_{i,j} \cdot X'_{l,j} = \sum_{j=1}^B 2(WX')_{i,j} \cdot (X')_{j,l}^T = \\ &2row_i(WX') \cdot column_j((X')^T) = 2(WX'(X')^T)_{i,l} \end{aligned}$$

$$\text{or } \frac{dv_i}{dW} = \begin{bmatrix} 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 2(WX'(X')^T)_{i,1} & 2(WX'(X')^T)_{i,2} & \dots & 2(WX'(X')^T)_{i,d} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix}$$

$$\text{Hence } P_{i,i} \frac{df(v_i)}{dW} = P_{i,i} \frac{df(v_i)}{dv_i} \cdot \frac{dv_i}{dW} = P_{i,i} \cdot \frac{-1}{2} (v_i + \epsilon)^{-3/2} \cdot \frac{dv_i}{dW} = \begin{bmatrix} - & 0 \cdot row_1(WX'(X')^T) & - \\ & \vdots & \\ - & -P_{i,i}(v_i + \epsilon)^{-3/2} \cdot row_i(WX'(X')^T) & - \\ & \vdots & \\ - & 0 \cdot row_k(WX'(X')^T) & - \end{bmatrix}$$

$$\text{Similarly, we get that } \frac{dL_2}{dW} = \sum_{i=1}^k P_{i,i} \frac{df(v_i)}{dW} = \begin{bmatrix} -P_{1,1}(v_1 + \epsilon)^{-3/2} \cdot row_1(WX'(X')^T) \\ -P_{2,2}(v_2 + \epsilon)^{-3/2} \cdot row_2(WX'(X')^T) \\ \vdots \\ -P_{k,k}(v_k + \epsilon)^{-3/2} \cdot row_k(WX'(X')^T) \end{bmatrix}$$

$$\text{Let } D' = \begin{bmatrix} P_{1,1}(v_1 + \epsilon)^{-3/2} & 0 & \cdots & 0 \\ 0 & P_{2,2}(v_2 + \epsilon)^{-3/2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & P_{k,k}(v_k + \epsilon)^{-3/2} \end{bmatrix} \text{ Then}$$

$$\frac{dL_2}{dW} = -D'W X'(X')^T(2). \text{ From (1), (2) we get } \frac{dL}{dW} = \frac{dL_1}{dW} + \frac{dL_2}{dW} = V \frac{dL}{dN} (X')^T -$$

$$D'W X'(X')^T$$

We update the parameters: $W \leftarrow W - c \frac{dL}{dW}$

6 Step 2: Feed forward in Part 2 and Step 5: Backpropagation happening inside Part 2

TUNEABLE PARAMETER: dropout rate α , learning rate c . **STEP 2: FEED FORWARD IN PART 2** Considering any feedforward from layer n to layer $n+1$. Let $H_n \in R^{k*B}$ be the output from layer n , let $W \in R^{k*k}$ be the weight matrix. Then we complete this feedforward layer similarly like step 1:

1. $M = WH_n \in R^{k*B}$ with W being a matrix that map H_n to M .

2. $N = DW H'_n$ being the batch normalization of M with $H'_n \in k * B$ such that

$$H'_n = \begin{bmatrix} | & | & & | \\ H_n^{(1)} & H_n^{(2)} & \dots & H_n^{(B)} \\ | & | & & | \end{bmatrix} \text{ with } E_H = \frac{1}{B} \cdot (H_n^{(1)} + H_n^{(2)} + \dots + H_n^{(B)})$$

and $H_n^{(i)} = H_n^{(i)} - E_H$ for all i in $1, 2, \dots, B$ and $H_n^{(i)}$ being the i -th column of H_n

$$\text{and } D \in k * k \text{ such that } D = \begin{bmatrix} f(v_1) & 0 & \cdots & 0 \\ 0 & f(v_2) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & f(v_k) \end{bmatrix}$$

with $f : R \rightarrow R$ such that $f(x) = (x + \epsilon)^{-1/2}$ and $v_i = \sum_{j=1}^B (W_i H_n^{(j)})^2$ being the variance of row i of M .

3. Affine transformation: $N' = \gamma N + \beta$

4. ReLU: $P = \text{ReLU}(N')$

5. Dropout: $H_{n+1} = \text{Dropoutwithrate}\alpha(P) \cdot (1 - \alpha)$

STEP 5: BACKPROPAGATION IN PART 2 At this step, by backpropagation in earlier steps we get the value of $\frac{dL}{dH_{n+1}}$
From the feedforward in step 2, the structure in part 2 is:

$$H_n \rightarrow H'_n \xrightarrow{VWH'_n} N \xrightarrow{\gamma N + \beta} N' \xrightarrow{\text{ReLU}(N')} P \xrightarrow{\text{Dropout}} H_{n+1}$$

1. Calculate $\frac{dL}{dP}$ in terms of $\frac{dL}{dH_{n+1}}$.

Similar to 1. in step 6, we have:

$$\left(\frac{dL}{dP}\right)_{i,j} = \frac{1}{1-\alpha} \cdot \left(\frac{dL}{dH_{n+1}}\right)_{i,j} \text{ if } (H_{n+1})_{i,j} \neq 0 \text{ and } \left(\frac{dL}{dP}\right)_{i,j} = 0 \text{ if } (H_{n+1})_{i,j} = 0$$

2. Calculate $\frac{dL}{dN'}$ in terms of $\frac{dL}{dP}$.

Similar to 2. in step 6, we have:

$$\left(\frac{dL}{dN'}\right)_{i,j} = \left(\frac{dL}{dP}\right)_{i,j} \text{ if } P_{i,j} \neq 0$$

$$\text{and } \left(\frac{dL}{dN'}\right)_{i,j} = 0 \text{ if } P_{i,j} = 0$$

3. Calculate $\frac{dL}{dN}$, $\frac{dL}{d\gamma}$, $\frac{dL}{d\beta}$ in terms of $\frac{dL}{dN'}$ and update β, γ .

Similar to 3. in step 6, we have:

$$\frac{dL}{d\gamma} = \sum_{i=1}^k \sum_{j=1}^B \frac{dL}{dN'_{i,j}} N_{i,j} = \text{Tr}\left(\left(\frac{dL}{dN'}\right)^T \cdot N\right)$$

$$\frac{dL}{d\beta} = \sum_{i=1}^k \sum_{j=1}^B \frac{dL}{dN'_{i,j}}$$

$$\frac{dL}{dN} = \gamma \cdot \frac{dL}{dN'}$$

$$\text{We update the parameters: } \gamma \leftarrow \gamma - c \frac{dL}{d\gamma}, \beta \leftarrow \beta - c \frac{dL}{d\beta}$$

4. Calculate $\frac{dL}{dW}$ in terms of $\frac{dL}{dN}$ and update the weight matrix W .

We proceed in a similar way to 4. in step 6: Let $U = WH'_n \left(\frac{dL}{dN}\right)^T$ and

$$D' = \begin{bmatrix} U_{1,1}(v_1 + \epsilon)^{-3/2} & 0 & \cdots & 0 \\ 0 & U_{2,2}(v_2 + \epsilon)^{-3/2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & U_{k,k}(v_k + \epsilon)^{-3/2} \end{bmatrix} \text{ then}$$

$$\frac{dL}{dW} = D \frac{dL}{dN} (H'_n)^T - D' W H'_n (H'_n)^T$$

$$\text{We update the parameters: } W \leftarrow W - c \frac{dL}{dW}$$

5. Calculate $\frac{dL}{dH'_n}$ in terms of $\frac{dL}{dN}$

In this case, W is fixed so D is also fixed since it relies entirely on W , or in other

words, $dW = dD = 0$. Therefore, $dN = (dD)WH'_n + D(dW)H'_n + DW(dH'_n) = DW(dH'_n)$

Therefore $dL = Tr((\frac{dL}{dN})^T dN) = Tr((\frac{dL}{dN})^T DW(dH'_n))$

Since D is a diagonal matrix, $D^T = D$ hence $dL = Tr((D\frac{dL}{dN})^T W(dH'_n)) = Tr((W^T D\frac{dL}{dN})^T (dH'_n))$ therefore $\frac{dL}{dH'_n} = W^T D(\frac{dL}{dN})$

6. Calculate $\frac{dL}{dH'_n}$ in terms of $\frac{dL}{dH'_n}$

Recall the definition of H'_n in step 2: $H'_n = \begin{bmatrix} | & | & & | \\ H_n^{(1)} & H_n^{(2)} & \dots & H_n^{(B)} \\ | & | & & | \end{bmatrix}$ with

$$E_H = \frac{1}{B} \cdot (H_n^{(1)} + H_n^{(2)} + \dots + H_n^{(B)})$$

and $H_n^{(i)} = H_n^{(i)} - E_H$ for all i in $1, 2, \dots, B$ and $H_n^{(i)}$ being the i -th column of H_n

Let $E = \begin{bmatrix} | & | & & | \\ E_H & E_H & \dots & E_H \\ | & | & & | \end{bmatrix}$ then $H'_n = H_n - E$

Notice that $E = \frac{1}{B} \cdot H_n \cdot \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & 1 & \dots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \dots & 1 \end{bmatrix} = H_n \cdot C$ with $C = \begin{bmatrix} 1/B & 1/B & \dots & 1/B \\ 1/B & 1/B & \dots & 1/B \\ \vdots & \vdots & \ddots & \vdots \\ 1/B & 1/B & \dots & 1/B \end{bmatrix}$

Hence let I_n be the identity matrix then $H'_n = H_n(I_n - C) = H_n I$

Then $dL = Tr((\frac{dL}{dH'_n})^T dH'_n) = Tr((\frac{dL}{dH'_n})^T (dH_n)I) = Tr(I(\frac{dL}{dH'_n})^T dH_n) =$

$Tr((\frac{dL}{dH'_n} I)^T dH_n)$ since I is a diagonal matrix.

Therefore $\frac{dL}{dH'_n} = (\frac{dL}{dH'_n})I = (\frac{dL}{dH'_n})(I_n - C)$.

7 Step 3: Feed forward and calculate the loss function in Part 3 and Step 4: Backpropagation happening inside Part 3

TUNEABLE PARAMETER: learning rate c . **STEP 3: FEED FORWARD AND CALCULATE THE LOSS FUNCTION IN PART 3**

Let $H_L \in R^{k*B}$ be the last hidden layer of this network. Let the weight matrix be $W \in R^{l*k}$, bias matrix be $C \in R^{l*B}$ such that $C = \begin{bmatrix} b & b & \dots & b \\ b & b & \dots & b \\ \vdots & \vdots & \ddots & \vdots \\ b & b & \dots & b \end{bmatrix}$

Let $M = WH_L + C \in R^{l*B}$, $Y = \text{softmax}(M)$ be the predicted output. Let's say the correct binary output for batch $1, 2, \dots, B$ be $i_1, i_2, \dots, i_B$, ($i_n \in \overline{1, l}$.) Then the loss function is $L = -\frac{1}{B}[(\ln(Y_{i_1,1}) + \ln(Y_{i_2,2}) + \dots + \ln(Y_{i_B,B})) = -\frac{1}{B}(g_1(M_{1,1}, M_{2,1}, \dots, M_{l,1}) + g_2(M_{1,2}, M_{2,2}, \dots, M_{l,2}) + g_B(M_{1,B}, M_{2,B}, \dots, M_{l,B}))]$.

With $g_n = \ln(\frac{e^{M_{i_n,n}}}{e^{M_{1,n}} + e^{M_{2,n}} + \dots + e^{M_{l,n}}}) = M_{i_n,n} - \ln(e^{M_{1,n}} + e^{M_{2,n}} + \dots + e^{M_{l,n}})$

STEP 4: BACKPROPAGATION IN PART 3

1. Calculate $\frac{dL}{dM}$ Consider any entry in column $n \in \overline{1, B}$ of M . We have:

$$\frac{dL}{dM_{i_n,n}} = -\frac{1}{B} \cdot \frac{dg_n}{dM_{i_n,n}}$$

$$= -\frac{1}{B} \cdot (1 - \frac{e^{M_{i_n,n}}}{e^{M_{1,n}} + e^{M_{2,n}} + \dots + e^{M_{l,n}}}) = \frac{1}{B} \cdot (\frac{e^{M_{i_n,n}}}{e^{M_{1,n}} + e^{M_{2,n}} + \dots + e^{M_{l,n}}} - 1).$$
Similarly $\frac{1}{B} \cdot \frac{e^{M_{i,n}}}{e^{M_{1,n}} + e^{M_{2,n}} + \dots + e^{M_{l,n}}}$ for any $i \in \overline{1, l}$ such that $i \neq i_n$.

Hence $\frac{dL}{dM}$ is calculated.

2. Calculate $\frac{dL}{dH_L}$ based on $\frac{dL}{dM}$

In this case W, C is fixed or $dW = dC = 0$ or $dM = W(dH_L) + (dW)H_L + dC = W(dH_L)$.

We have: $dL = \text{Tr}((\frac{dL}{dM})^T dM) = \text{Tr}((\frac{dL}{dM})^T W(dH_L)) = \text{Tr}((W^T \frac{dL}{dM})^T dH_L)$
therefore $\frac{dL}{dH_L} = W^T(\frac{dL}{dM})$

3. Calculate $\frac{dL}{db}$ based on $\frac{dL}{dM}$ and update b .

$$\frac{dL}{db} = \sum_{i=1}^l \sum_{j=1}^B \frac{dL}{dM_{i,j}} \cdot \frac{dM_{i,j}}{db} = \sum_{i=1}^l \sum_{j=1}^B \frac{dL}{dM_{i,j}} \cdot \frac{d((\sum_{t=1}^k W_{i,t}(H_L)_{t,j}) + b)}{db} = \sum_{i=1}^l \sum_{j=1}^B \frac{dL}{dM_{i,j}}$$

We update the bias b : $b \leftarrow b - c \frac{dL}{db}$

4. Calculate $\frac{dL}{dW}$ based on $\frac{dL}{dM}$ and update W .

In this case H_L, b is fixed or $dH_L = dC = 0$ or $dM = (dW)H_L$.

We have: $dL = \text{Tr}((\frac{dL}{dM})^T dM) = \text{Tr}((\frac{dL}{dM})^T (dW)H_L) = \text{Tr}(H_L(\frac{dL}{dM})^T dW) = \text{Tr}((\frac{dL}{dM})H_L^T)^T dW$ therefore $\frac{dL}{dW} = (\frac{dL}{dM})H_L^T$

We update the weight matrix W : $W \leftarrow W - c \frac{dL}{dW}$

8 But what if we don't use batch normalization?

For simple tasks like MNIST digit recognition, we don't actually need batch normalization for our neural network.

In this case, the math becomes extremely simple. Recall that in a neural network, the two main parts are the hidden layer to the loss function, and the hidden layer to another hidden layer. If we don't use batch normalization then the only part that change is between 2 hidden layers, as it becomes:

$H_n \xrightarrow{WH_n+C} N \xrightarrow{ReLU(N)} P \xrightarrow{Dropout} H_{n+1}$ with C being bias matrix with all entries being b .

The backpropagation step from H_{n+1} to P and from P to N is basically the same as above:

Calculate $\frac{dL}{dP}$ in terms of $\frac{dL}{dH_{n+1}}$.

$$(\frac{dL}{dP})_{i,j} = \frac{1}{1-\alpha} \cdot (\frac{dL}{dH_{n+1}})_{i,j} \text{ if } (H_{n+1})_{i,j} \neq 0 \text{ and } (\frac{dL}{dP})_{i,j} = 0 \text{ if } (H_{n+1})_{i,j} = 0$$

Or in other terms, denote $\odot$ as element-wise multiplication of 2 matrices, let $H_{n+1} = \text{Dropout}(P) = \frac{1}{1-\alpha} \cdot \text{dropoutMatrix} \odot P$ then $\frac{dL}{dP} = (\frac{dL}{dH_{n+1}} \odot \text{dropoutMatrix}) \cdot \frac{1}{1-\alpha}$

Calculate $\frac{dL}{dN'}$ in terms of $\frac{dL}{dP}$.

$$(\frac{dL}{dN'})_{i,j} = (\frac{dL}{dP})_{i,j} \text{ if } P_{i,j} \neq 0$$

$$\text{and } (\frac{dL}{dN'})_{i,j} = 0 \text{ if } P_{i,j} = 0$$

Now having calculated $\frac{dL}{dN}$, we will calculate $\frac{dL}{dW}$ and $\frac{dL}{dH_n}$ based on $\frac{dL}{dN}$:

$$\text{When } W, b \text{ is fixed: } dL = Tr((\frac{dL}{dN})^T dN) = Tr((\frac{dL}{dN})^T W(H_n)) = Tr((W^T \frac{dL}{dN})^T H_n)$$

$$\text{therefore } \frac{dL}{dH_n} = W^T (\frac{dL}{dN})$$

$$\begin{aligned} \text{When } H_n, W \text{ is fixed: } \frac{dL}{db} &= \sum_{i=1}^k \sum_{j=1}^B \frac{dL}{dN_{i,j}} \cdot \frac{dN_{i,j}}{db} = \sum_{i=1}^k \sum_{j=1}^B \frac{dL}{dN_{i,j}} \cdot \frac{d((\sum_{l=1}^k W_{i,l}(H_n)_{l,j}) + b)}{db} = \\ &= \sum_{i=1}^k \sum_{j=1}^B \frac{dL}{dN_{i,j}} \end{aligned}$$

$$\text{We update the bias } b: b \leftarrow b - c \frac{dL}{db}$$

$$\begin{aligned} \text{When } H_n, b \text{ is fixed: } dL &= Tr((\frac{dL}{dN})^T dN) = Tr((\frac{dL}{dN})^T (dW) H_n) = Tr(H_n (\frac{dL}{dN})^T dW) = \\ &= Tr((\frac{dL}{dN} H_n^T)^T dW) \text{ therefore } \frac{dL}{dW} = \frac{dL}{dN} H_n^T. \end{aligned}$$

$$\text{We update weight matrix } W: W \leftarrow W - c \frac{dL}{dW}.$$

What if we use L2 regularization?

In this case, the only change is in the backpropagation process, specifically to weight matrix only, since now $L_{new} = L_{original} + \lambda(\sum w^2)$ with w being every entry of every weight matrix of the model.

Now, consider any weight matrix $W \in R^{a \times b}$ in the model then $\frac{dL_{new}}{dW} =$

$$\frac{dL_{original}}{dW} + \lambda \frac{d(\sum_{i=1}^a \sum_{j=1}^b (W_{i,j})^2)}{dW} = \frac{dL_{original}}{dW} + \lambda \cdot 2W$$